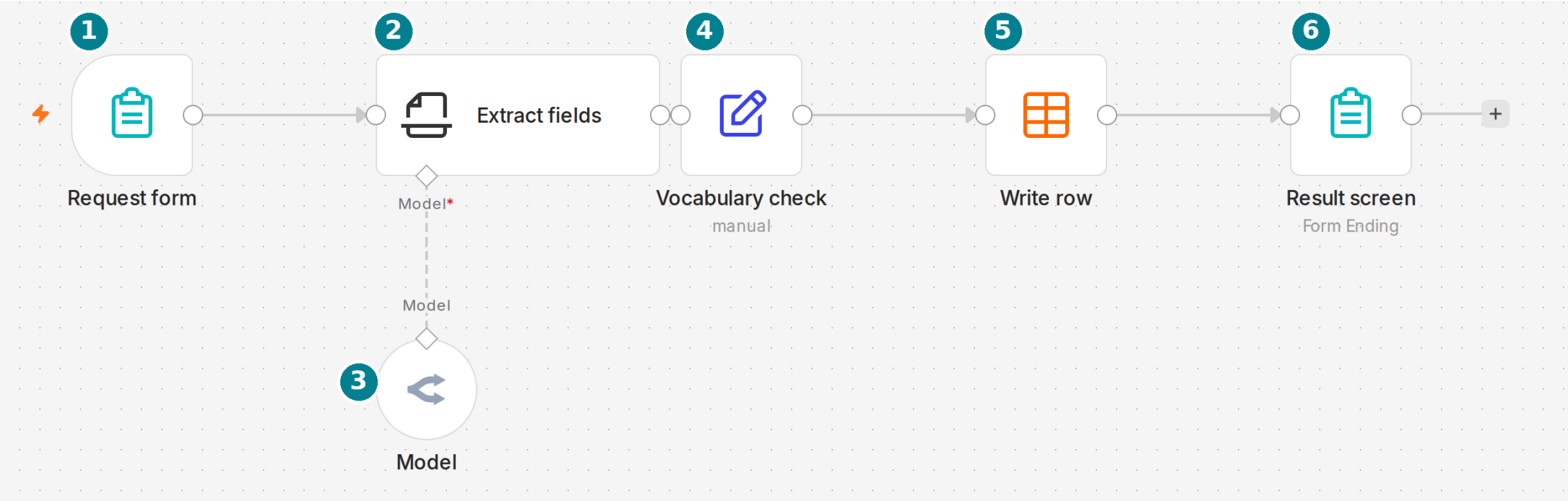
# Where the model decides, and where code must

Large-language-model workflows: the second session of the arc.

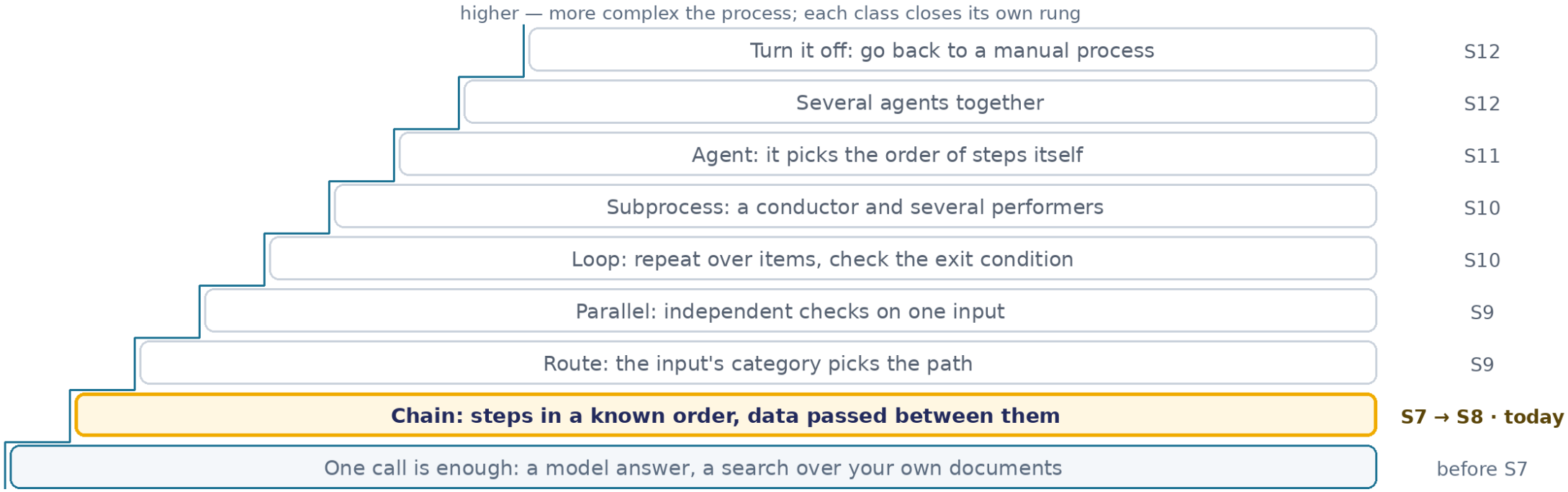
# Starting state: the workflow from last session




1 form — the process's entry point · 2 extraction — pulls values out of the text · 3 model — the second node of the step
4 lookup check against the reference list · 5 write the row · 6 result screen — prints the whole trail

**This is a real n8n canvas, not a diagram. We have a workflow for classifying incoming requests — six nodes, the same order, the same names as at the end of last session.**

# Same rung: chain




**The gold bar sits in the same place as last time. Today's four cases are not a climb higher — they are what happens inside one and the same rung when new steps get added to the chain.**

# The session's axis: where the model decides, where code must

| LOOKS RELIABLE | TURNED OUT TO BE |
| --- | --- |
| the message reached the coordinator, the chain got shorter and cheaper without an LLM | |
| a repeat request about the same question came in — handled as new | |
| the prompt says "don't refund more than amount X" | |
| the image got assembled and sent to Telegram | |

The right column won't close all at once at the end — one row at a time, right after the case that closes it. And each time, not by my telling it, but by what we see on screen.

# Case 1. Notifications

*"Send me one line in Telegram: who wrote and about what — so I don't have to check the table for every request."*

**Уведомления** | Проверка на повтор | Одобрение возврата | Карточка в Telegram

# The result screen and the multi-topic request

**The training center, the same process that went live last session: form → field extraction → write to a table with topic, extra topics, and urgency → result screen. Olga, the administrator, works with this screen — twenty requests a day, and she clears most of them calmly straight off the result screen: the topic and urgency are already there, and who to hand it to is obvious at a glance.**

1. Every few requests, the text doesn't come out shorter than usual: the person mentioned something beyond the main question — asked about payment, asked to reschedule a class, asked about the timetable. The "extra topics" field in the table catches this and records it honestly.
2. Olga doesn't open the result screen for every request right away — there are twenty a day, and opening each one the moment it's ready would be exactly the manual work the process was supposed to remove. She checks in batches, several times per shift.
3. If a multi-topic request arrives between checks, it sits in the table with the right fields — and looks no different from an ordinary request until Olga opens the screen herself and reads that exact row.
4. By evening, on a busy day, she admits honestly: she noticed some multi-topic requests not the moment they arrived, but at the next batch check — sometimes hours later.

**What the training center pays for the delay. A multi-topic request is almost always more urgent than an ordinary one: the person isn't just asking about the course, they also want to move a paid class to today, or clarify something that has to happen before the end of the day. A batch check every few hours turns exactly that urgency into an ordinary queue.**

**Olga states her wish with total precision: "Send me one line in Telegram for every request like that — who wrote, on what topic, and what else they mentioned — so I don't have to go check the result screen because of one multi-topic request among twenty ordinary ones."**

# How to assemble the notification from three values

**How do you assemble the notification text from these three values?**

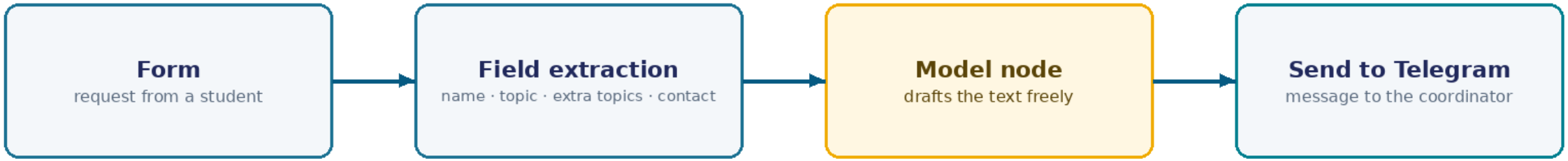
- **the model writes the whole notification text freely**
- **we fill in a ready-made message form entirely by algorithm — substitution only**
- **whatever the algorithm can close, it closes; whatever requires interpreting the request text, we call the model for that separately**
- **we take the algorithmically assembled text as the base and add one phrase of tone from the model on top**

# What each action actually does

| ACTION | HOW IT WORKS | ADVANTAGE | DRAWBACKS |
|---|---|---|---|
| **the model writes the whole text** | gets three already-extracted values and freely composes the notification text, with no fixed form | the text sounds alive, not like a canned phrase — it can write like a human | the values are already extracted — there's nothing to interpret; the text is re-assembled on every run, with a risk of slipping in a detail that wasn't in the request |
| **the algorithm fills in the whole form** | substitutes three already-structured values into a template text fixed once, with no call to the model | the result is deterministic: it matches the given text example on any request, not just a lucky one | no room for variability: if the message form ever needs to change in meaning, not just in values, the expression has to be rewritten by hand |
| **algorithm + model for what requires interpretation** | structured values are substituted by the algorithm; whatever stayed unstructured and requires interpreting the request text is passed to the model separately | would work honestly for a genuinely mixed case — where at least one value really requires interpretation, not just is already known | for this specific need, there's nothing to interpret: all three values, including urgency, were already extracted structurally last session — there's simply no work left for the model here |
| **algorithm + model's tone on top** | an algorithmic backbone of text is assembled by substitution, and the model adds one short phrase of tone or comment on top | the scope of freedom is smaller than in the first action — the risk and cost of calling the model are correspondingly lower | the same kind of risk as the first action, just smaller in scale: the model still generates text and can slip in a detail that wasn't there |

# Architecture: the model-step composes the text

## Solution with an LLM: the model drafts the text



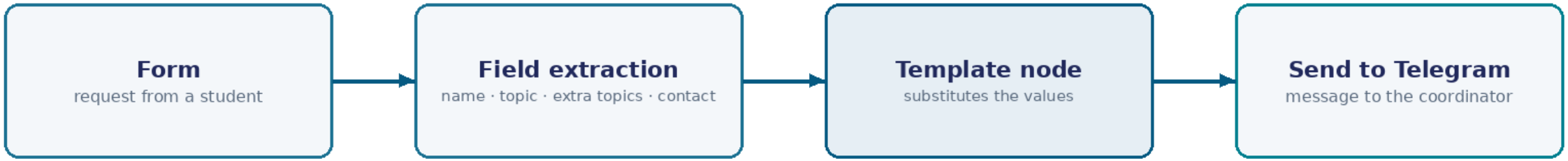

**Risk: the model can add a detail that was not in the request.**

Plus the cost and latency of calling the model on every run.

**Four blocks: form → field extraction → a step that calls the model on every run → Telegram. The decision about the text's form is made anew on every request.**

# Architecture: the template-step substitutes values

## Solution without an LLM: a template fills in the values

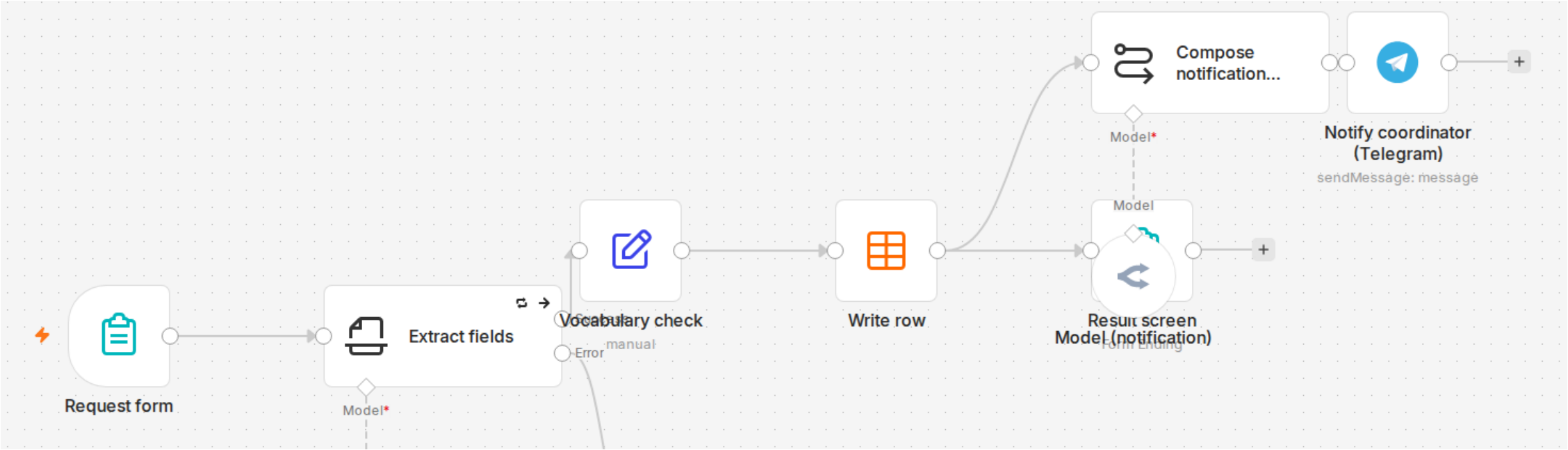



**No model call — no risk of an invented detail.**

The same message shape, assembled once in a template.

**The same chain: form → field extraction → substituting values into a fixed form → Telegram. The decision about the text's form was made once, in advance, not on every run.**

# The whole path on the rig: the model version




**Request form → field extraction (calls the model) → vocabulary check → write row → "Compose notification…" (calls the model a second time, the caption cut off by an ellipsis — that's how the n8n interface behaves with a narrow card) → send to the coordinator in Telegram.**

# Node settings: the prompt of the model version




**Node "Compose notification (LLM)", Parameters tab. The Prompt (User Message) field (real screen, text in Russian — see Assertion above): «Напиши короткое дружелюбное уведомление координатору учебного центра в Telegram о новой многотемной заявке. Имя заявителя: `{{ $('Request form').item.json['Имя'] }}`. Основная тема: `{{ $('Vocabulary check').item.json.topic }}`. Также в заявке упомянуто: `{{ $('Vocabulary check').item.json.extra_topics }}`. Пиши по-человечески, одно-два предложения.» — in English, what this instructs: "Write a short, friendly Telegram notification to the training center's coordinator about a new multi-topic request. Requester's name: {{ … }}. Main topic: {{ … }}. The request also mentioned: {{ … }}. Write it the way a human would, one or two sentences."**

**The Require Specific Output Format and Enable Fallback Model toggles — both off: no fixed answer structure, no fallback model if the call fails.**

# What the message looks like on Olga's phone

**Telegram, chat "Training center notifications."**

**New request from Maria: topic "course," also mentioned: payment.**

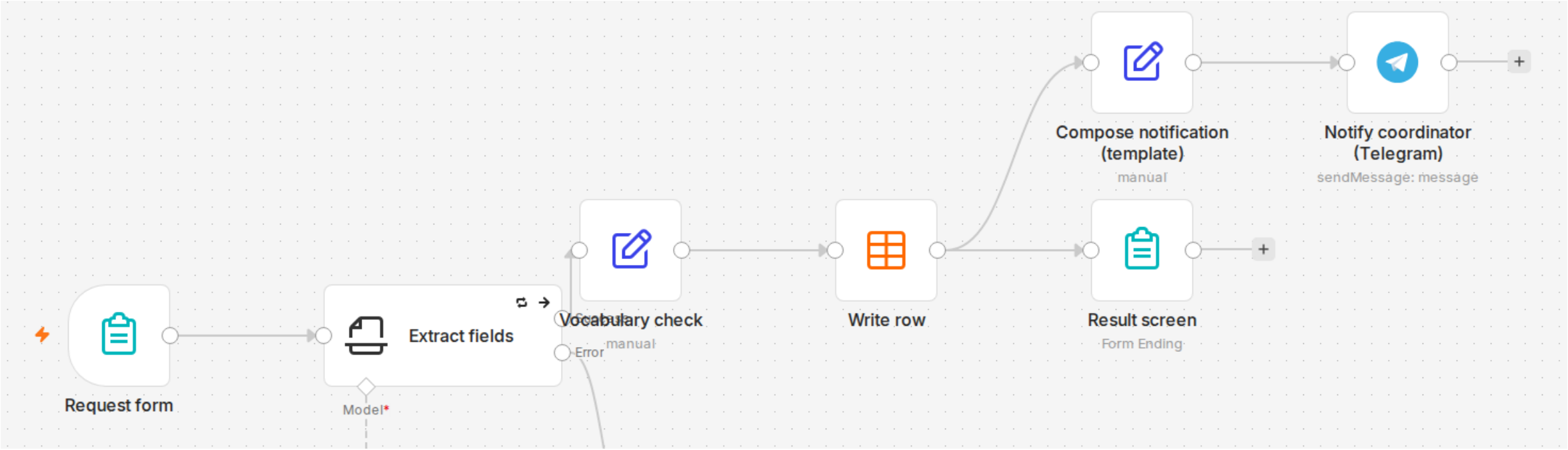
**Exactly what Olga described in the previous scene, word for word: who wrote, on what topic, and what else they mentioned — one line, with no need to open the result screen.**

# What two real runs actually produced

| REQUEST TEXT | WHAT CAME OUT AND WHY |
| --- | --- |
| Maria: "I want to sign up for the analytics course and also ask about payment, please call back today" | Delivered: "New request from Maria: topic 'course,' also mentioned: payment." Free-form text, but it matched the goal in meaning — the model had enough from the instruction and the three values; interpreting was genuinely simple here. |
| Oleg: "I want to move my class to Friday and ask about course payment" | Delivered: "Hi, **[Coordinator's name]**! We have a new multi-topic request from Oleg about moving a class and about payment." The model inserted a literal fragment of its own instruction template — words addressed to itself, not to Olga. The node didn't fail, the message was delivered, the status was green: the mismatch is visible only in the content. |

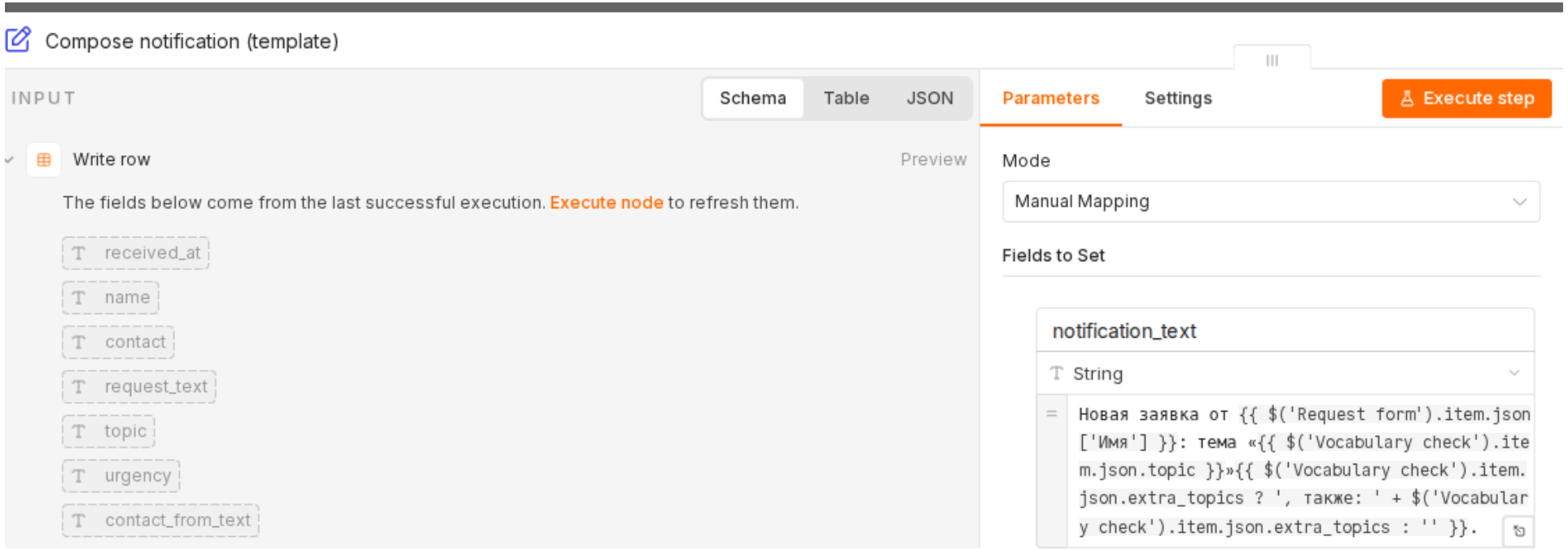
**A format check doesn't catch this. Both runs are technically successful: both nodes completed, both messages were delivered. The only way to notice the difference is to read the actual text a real person received.**

# The whole path on the rig: the template version




**Request form → field extraction (calls the model) → vocabulary check → write row → "Compose notification (template)" (the caption reads in full, no model call) → send straight to the coordinator in Telegram.**

# Node settings: the expression of the template version




**Node "Compose notification (template)" (`Set` / `Edit Fields`), Mode — Manual Mapping. Field `notification_text` (String), real screen, Russian field names and output text — see Assertion above: «Новая заявка от `{{ $('Request form').item.json['Имя'] }}`: тема «`{{ $('Vocabulary check').item.json.topic }}`»`{{ $('Vocabulary check').item.json.extra_topics ? ', также: ' + $('Vocabulary check').item.json.extra_topics : '' }}`.» — in English, what this produces: "New request from {name}: topic '{topic}'{, also: {extra topics}, if present}."**

*"Include Other Input Fields" — on, "Input Fields to Include" — All: the rest of the request's fields pass through the chain unchanged.*

# Not every text output is generation

| | |
|---|---|
| **PRINCIPLE** | **IF ALL THE NEEDED VALUES ARE ALREADY STRUCTURED AND THE MESSAGE FORM DOESN'T CHANGE IN SUBSTANCE, THIS IS A FORMATTING TASK, NOT A MODEL TASK.** |
| **Task class** | Assembling a message from already-extracted values in a fixed form. |

# When you need the model, when a template is enough

NUANCE

**An extra step is a risk, not just a cost**

**A template isn't permanently safe**

WHAT IT MEANS

Every model call is a point where a hallucination can appear — even in a trivial formatting task.

If the message form ever needs genuine variability (different languages, different levels of detail), the template will have to be rewritten by hand for every new case — a different kind of cost, an engineering one.

**Criterion. The model is needed when the content genuinely varies, or when a value requires interpreting text that wasn't interpreted at an earlier step. A template is enough when the message has one form and every value is already structured.**

**The first line of the axis is closed. The message really did reach the coordinator — and the chain turned out shorter and cheaper without the model node. The LLM here was simply an extra step: the template produced the same result, without its cost or its risk.**

# Case 2. Duplicate Check

*"Isn't this the same question he wrote in two hours ago?"*

# Requests Duplicate

1. Monday morning. Olga, the training center's administrator, opens the requests table and sees a question from a client: "when can I pay for the class?" She answers.
2. Two hours later — a second request from the same contact: "how do I pay for the course". In meaning, it's the same question. Olga doesn't recognize the rephrasing and processes it as new: answers again.
3. The client gets two different answers from the center two hours apart — and it looks as if the first answer got lost.

**This isn't rare. By Olga's own estimate, such repeats happen about 10 times a day: someone forgot they'd already asked, someone didn't see the answer and asked again in different words.**

**The cost of the mistake isn't abstract. Double the administrator's work — plus the client's distrust: "don't they see I already asked?"**

# How to Build a Duplicate Check

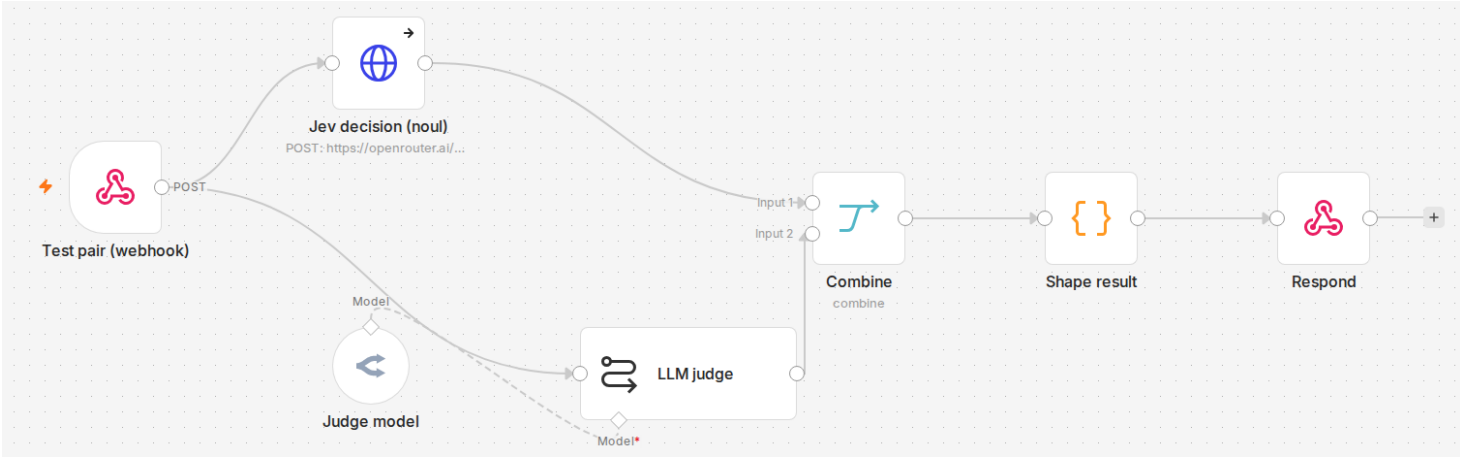
**How do you build a duplicate check?**

- compare the texts literally — exact match after normalization
- compare the texts by shared words — the classic vector approach with a threshold
- ask a general-purpose model whether it's the same question, just phrased differently
- ask a specialized question-comparison service instead of a general-purpose model
- check exact match cheaply first, and only pass through what's left

# What Each Scheme Caught

| SCHEME | CORRECT OUT OF 20 | WHAT IT CAUGHT | WHAT IT MISSED / GOT WRONG | COST AND SPEED |
|---|---|---|---|---|
| **Hash (exact match)** | **15/20** | all 5 repeats, all 10 different ones | all 5 rephrasings (0/5) | cheap and instant — no model call |
| **TF-IDF + cosine** (threshold 0.3) | **13/20** — worse than hash | repeats, some of the different questions | the same 5 rephrasings + 2 false positives on close topics (hash doesn't make this mistake) | cheap and instant — no model call |
| **Judge model** (gpt-4o-mini) | **17/20** | all 10 duplicates | 3 false positives, including a pair with no shared words | ~$0.15/M in · $0.60/M out — for the case (≈300/mo) ≈$0.006/mo (price-list estimate) |
| **jev service** | **20/20** | all 20, zero errors | clean split, no overlap | $0.042/M in, $0 out — for the case ≈$0.003/mo (price-list estimate) |

**Honest finding. TF-IDF misses every rephrasing, adds 2 false positives where hash never errs. jev scored best — accurate, cheap — but one 20-pair run can't justify replacing a working architecture. At this case's volume (≈10/day) both paid options cost under a cent a month: accuracy decides.**



# Solution: Hash First, jev on the Expensive Step

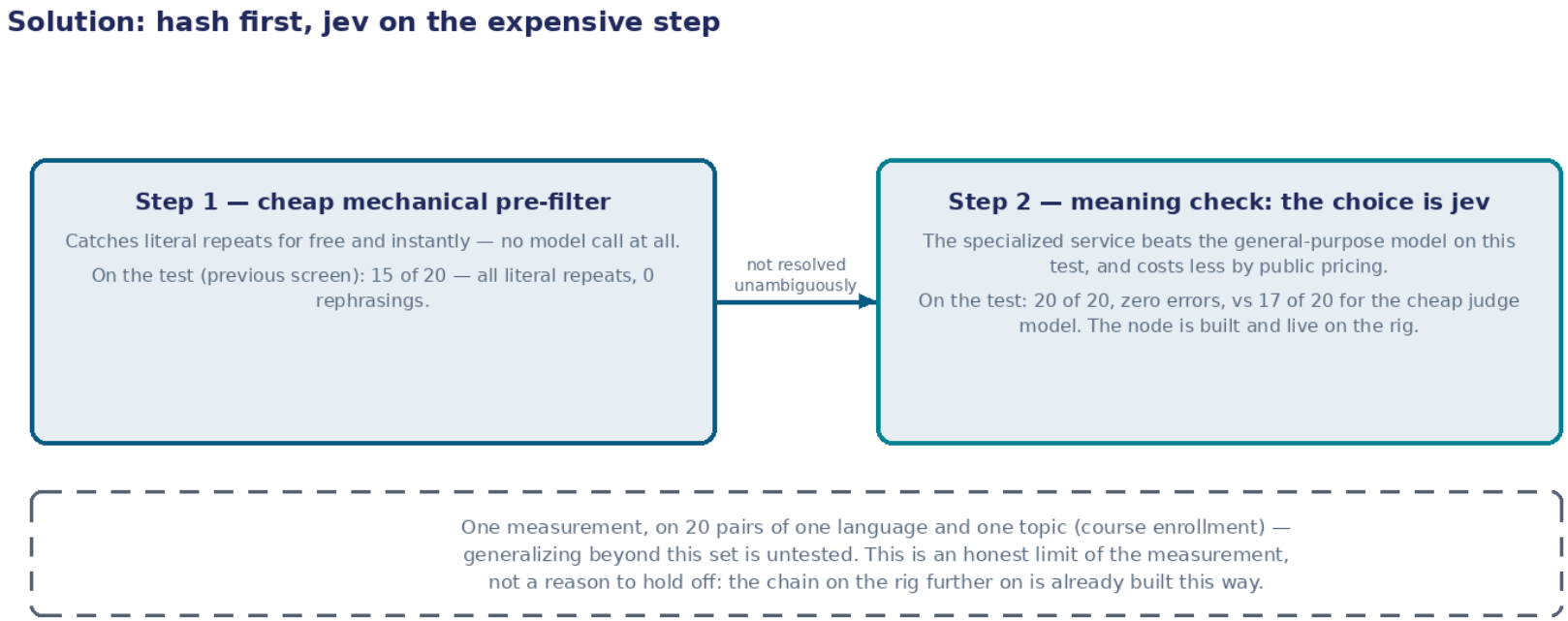
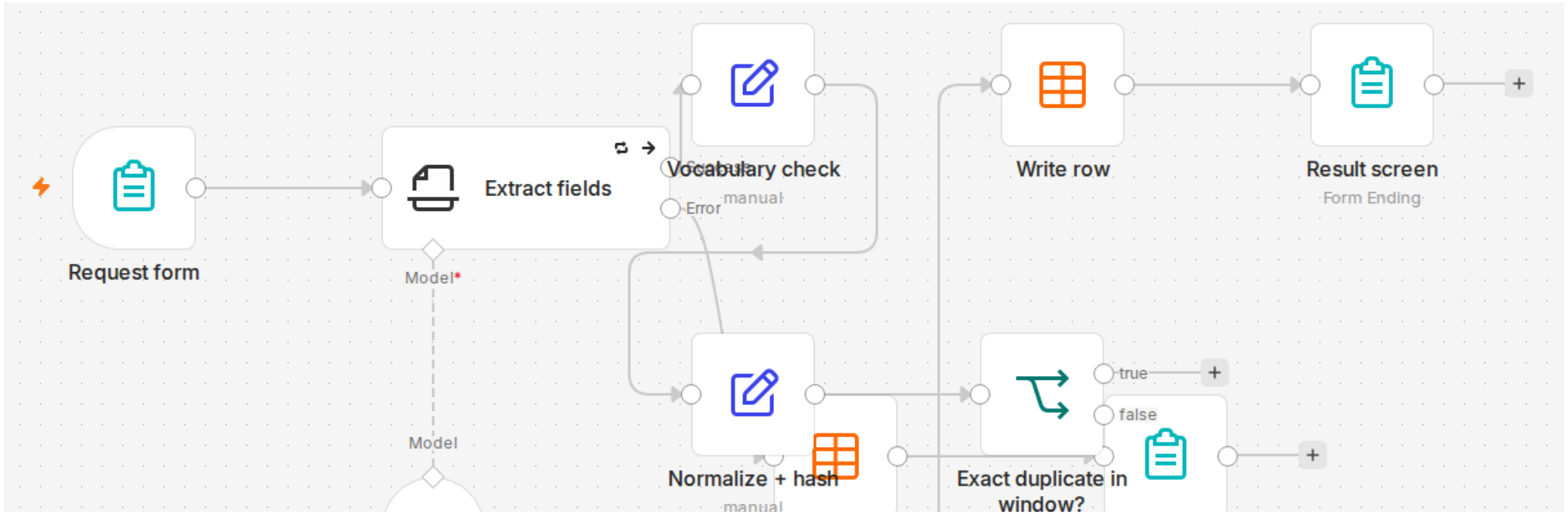



**Figure: step 1 — the cheap mechanical pre-filter, takes on literal repeats for free. Step 2 — the meaning check, the measurement's choice is the specialized service (jev), more accurate and cheaper than the general-purpose model on this test. The node is built and live on the rig.**

**Honest caveat — right on the figure. One measurement, on 20 pairs of one language and one topic (course enrollment). Generalizing beyond this set is untested — this is a boundary of the measurement, not a reason not to deploy the solution.**

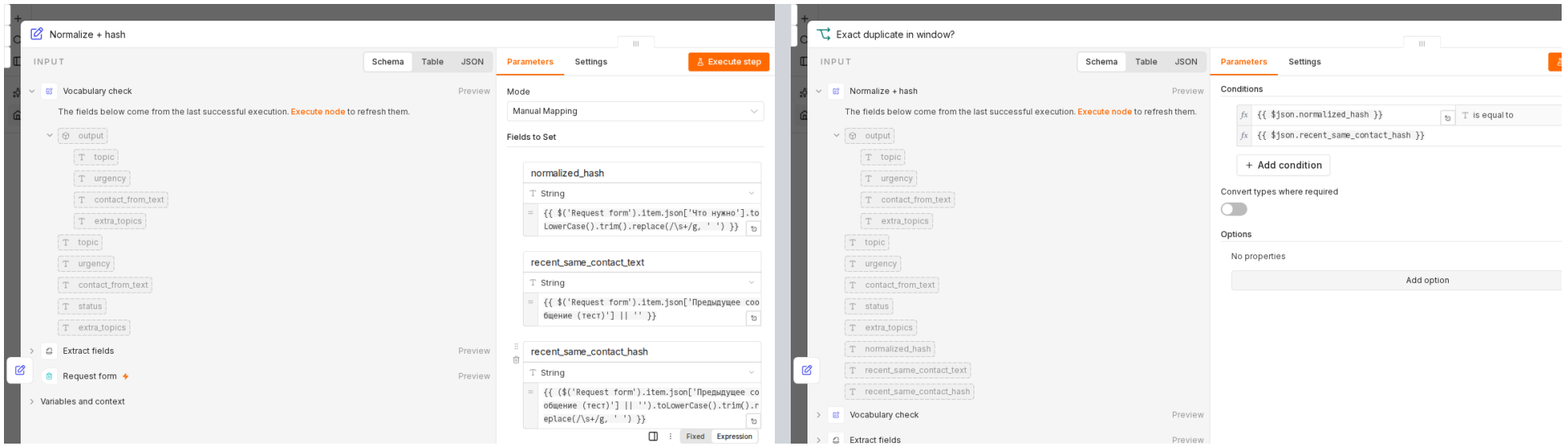
# Configuration: Normalization and Exact Match




Node `Normalize + hash` (`Set`). Takes the request's text, lowercases it, trims the edges, collapses repeated internal spaces: `normalized_hash` = `text.toLowerCase().trim().replace(/\s+/g, ' ')`. Alongside — the same calculation for the same contact's recent message.

Node `Exact duplicate in window?` (`IF`). Condition: `normalized_hash` is strictly equal to the recent request's hash — a string comparison, both values already lowercased. Match → a repeat is found, caught for free, no record written. No match → the request moves on down the chain.

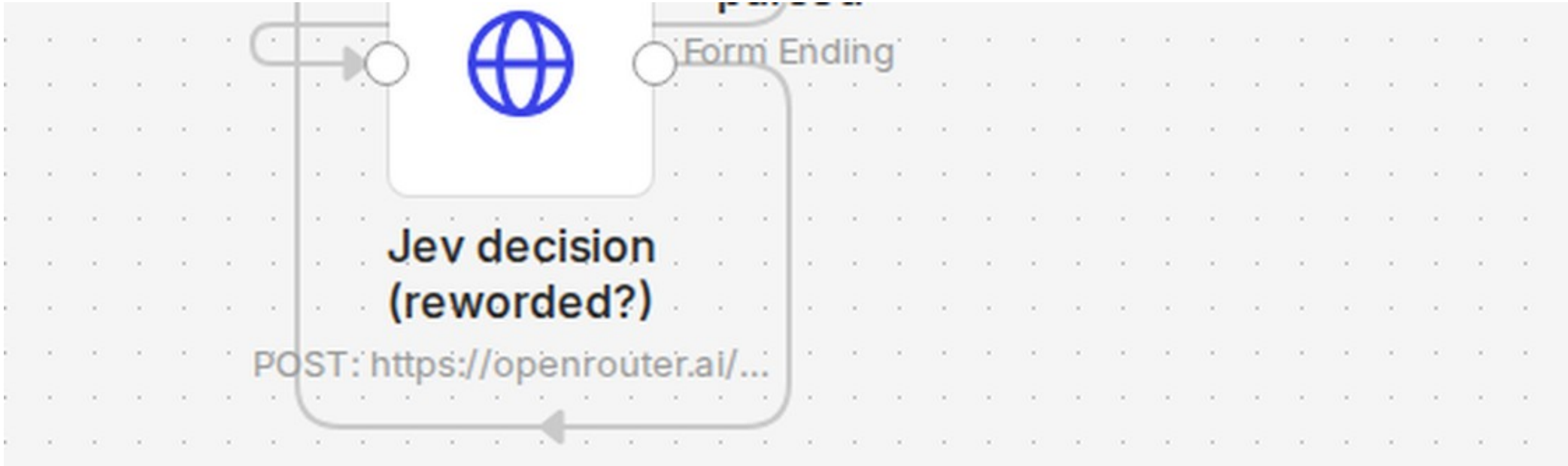
# Node Settings: The Parameters Screen




- Node `Normalize + hash` (`Set`, `Manual Mapping` mode). Three fields with expressions: `normalized_hash` — the current request's normalized text (`$('Request form').item.json['Что нужно'].toLowerCase().trim().replace(/\s+/g, ' ')` — the field name `'Что нужно'` is the real, literal name of this field on the live Russian-language intake form, not an artifact of translation); `recent_same_contact_text` — the same contact's recent message as is, unchanged; `recent_same_contact_hash` — that same text, normalized by the same formula. "Include Other Input Fields" is on (All) — the rest of the request's fields pass through untouched.

- Node `Exact duplicate in window?` (`IF`). One condition: `{{ $json.normalized_hash }}` is equal to `{{ $json.recent_same_contact_hash }}` — the previous node's third field compared against its first. "Convert types where required" is off: the strings are compared as-is, with no hidden type coercion.

# Configuration: A Structured Request Instead of a Text Prompt




Node `Jev decision (reworded?)` (`httpRequest`). `POST .../api/alpha/decisions`, model `typesafe/jev-1.13`. The question type is `noul`: "yes/no" with a probability, not free-form text. The criteria are given explicitly: "yes" — the same question in different words, "no" — a different question, even if the topics or some words look similar.

Result on 20 real pairs. Duplicates — jev_noul 0.70–0.97. Different questions — 0.02–0.08. Threshold ≥ 0.5 → duplicate, a clean gap across 0.08–0.70 — the score is 20 out of 20.

# What Olga Sees on a Repeat Request — and What She Doesn't

**The requests table, that same morning.**

**Row 1 — "when can I pay for the class?" — an ordinary request.**

**(Two hours later the same text, word for word — there's no second row. The repeat was stopped before it reached the table.)**

**Row 2 — "how do I pay for the course" — logged, and looks exactly like an ordinary new request.**

**For the literal repeat, the scene's problem is solved completely: it simply never reaches Olga. For the very same rephrasing the case started with — it gets logged as-is, as a second row, no different from the first.**

# Mechanics and Meaning — Different Tools, Not Versions of Each Other

**The systemic takeaway. Exact match stays in the final solution — but as the first cheap filter, saving 5 of 20 model calls for free, not as an alternative to the LLM.**

**Further in the course. In Seminar 10, this same mechanic — comparing the meaning of requests — scales up into searching for an answer in a knowledge base.**

# Nuances of the Duplicate Check

1 **The time window — a parameter with a cost on both sides.** Too short a window will miss a repeat spread a day or two apart. Too long a window will start confusing a person's different, consecutive requests with a repeat of one request.

1 **The LLM judge has its own error asymmetry, not zero** (Wataoka, Takahashi, Ri — NeurIPS 2024 Safe GenAI Workshop, arXiv:2410.21819, self-preference bias): a judge model is systematically non-neutral, even when formally answering "yes" or "no". Our own test confirmed this directly: 3 false positives out of 20, and not all of them come from word overlap in the questions.

1 **"What counts as a duplicate" is a product decision, not just a technical one.** Sometimes "the same question in different words" is worth treating as a new request — the person is waiting for an answer more urgently the second time. Not every technical repeat has to collapse into one in the interface.

# When Exact Match Is Enough

### EXACT MATCH IS ENOUGH

repeats are mostly literal, like Boris's first question

you need a free, instant filter on the whole request stream

the classic vector approach isn't worth adding "just in case" — the measurement showed a result WORSE than hash, not the same

### MEANING CHECK IS NEEDED ON TOP

rephrasings are expected, like his second question

the cost of a missed duplicate is higher than the cost of one model call on the borderline cases

the frequency of the model's own errors is a measured cost (3 false positives out of 20 for us), not zero, and it has to be budgeted for too

neither hash nor the classic vector approach solves rephrasing in Russian on their own, in full

the specific meaning-check tool is also a choice: a cheap general-purpose model (17/20 on our test) or the specialized service jev (20/20) — not something to guess by default, it needs its own check

**Closes the axis's row. It looked reliable: a request about the same question came in again — handled as new. What it turned out to be: without a duplicate check, every repeat request restarts the chain from scratch; neither hash nor the classic vector approach solves this entirely for live Russian text — a model is needed, with its own, measured price for being wrong.**

# Case 3. Refund approval

*"Automatic refund — up to a set amount. Above the threshold — only through the manager."*

Уведомления | Проверка на повтор | **Одобрение возврата** | Карточка в Telegram

# Version 1: a continuation of the classification process

**The same request form we've already used in the previous cases: a person describes the situation in their own words — "please refund 2,500 rubles for a missed class."**

1. A person writes a request in free text — it contains both the ask and the amount.
2. The process that in Seminar 7 determined the topic of a request now goes one step further here: it reads the text and decides on its own — approve the amount automatically, or pass it to the manager (model `qwen/qwen-2.5-7b-instruct`).
3. Decision "approve" — the money moves to the client's card automatically, with no human involved.
4. Decision "escalate" — the request goes to the manager, and the manager decides from there.

**What today's trust rests on. The version has been running for a month. Nobody has complained that something was approved that should have been escalated, or the other way around. The team considers the rule reliable — on exactly that basis: nothing bad has been noticed.**

# How to make sure the threshold decision never errs

**How do you make sure the threshold decision is never made incorrectly?**

- write the rule more clearly in the prompt
- give the model a calculator tool for comparing amounts
- put an IF node after the amount is extracted
- make the amount an explicit numeric field on the form

# What each of the four actions delivers

| ACTION | HOW IT WORKS | ADVANTAGE | DRAWBACKS |
|---|---|---|---|
| **Write the rule more clearly in the prompt** | Reword the rule, explicitly separating "≤3,000 → approve" from ">3,000 → escalate" | Measured: version 1 erred 9/101 at the boundary; the clarified wording — **100/100** on the extended set | A fact about this specific set/model, not a structural guarantee — the decision is still made by text, not code; another wording or model could diverge again |
| **Give the model a calculator tool** | The model gets a number-comparison function instead of comparing "in its head," and decides itself whether to call it | In theory splits extraction and comparison into two operations | The model still extracts the amount from text first — the same probabilistic step — then probabilistically decides whether to call the tool: the probabilistic part doesn't disappear, it moves earlier. Measured: **56/100** — worst of all options; the calculator can't do comparisons (11/100 refusals), 30/100 answers don't reduce to a clean word, 9/100 errors scattered across the range |
| **Put an `IF` node after the amount is extracted** | The model still pulls out the amount, code compares it with the threshold — a condition, not a text call | Removes the probabilistic part from the decision point. Measured: **0/101** on the extended set, including boundary cases | The model still extracts the amount — an extraction error (not a comparison error) isn't closed by this node, though none turned up on this set |
| **Make the amount an explicit numeric field on the form** | Collect the amount as digits in a form field — never extract it from text at all | Removes the uncertainty earlier: there's nothing to extract, so there's no extraction-error risk | Works only if the form can be changed; an email/chat/other channel still requires extraction from text. The metric is applicability, not accuracy — there's no extraction step, so there's no accuracy to measure either |

**Bridge ahead. One action measurably fixed the specific finding, one measurably failed, one eliminates the risk structurally and is confirmed by measurement, one changes the input before the decision point and needs no accuracy metric at all. Which one is right here isn't decided by argument at the table — it's decided by the measurement you're already seeing in this table; the next beat shows how it was built.**

# Testing an idea: the jev service for extracting the amount

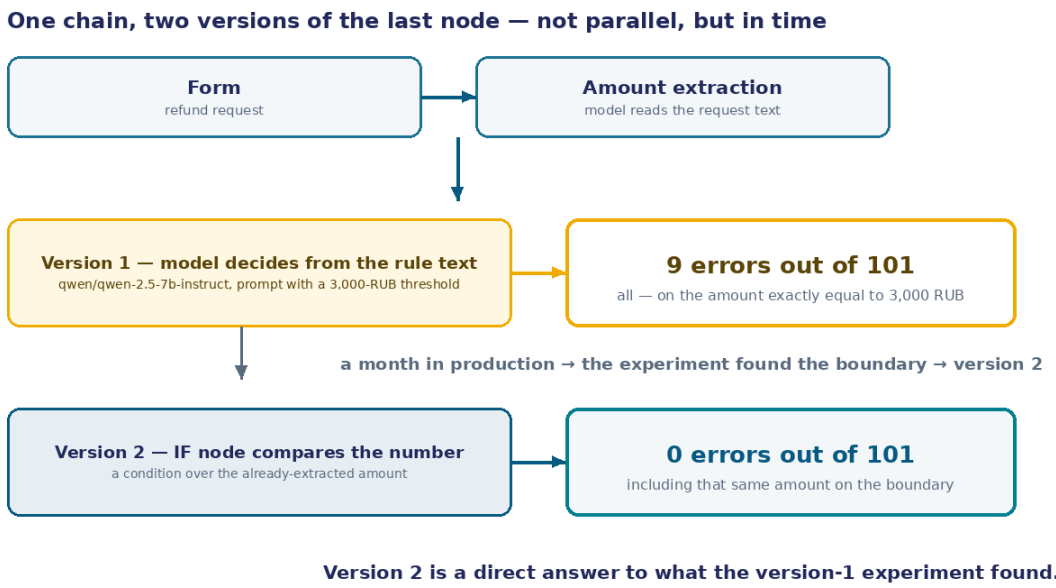
**The proposed idea. Pull the refund amount out not with a general-purpose model, but with the specialized jev service — the same one that, in Case 2, told a repeat question from a new one — and still leave the threshold comparison to code, the same `IF` node already chosen by version 2.**

**What the test showed. The jev service has exactly four response types: yes/no with a probability, a choice from a fixed list of options, a position on a predefined scale, a region on an image. An open-ended number is not among them. The decisive test: amounts of 2,999 and 3,001 rubles — on opposite sides of the real 3,000-ruble threshold — both came back with the same scale bucket, with full confidence. The service did not tell them apart. The only way to tell them apart would have been to build the bucket boundary at exactly 3,000 in advance — that is, to know the threshold at the moment of framing the question to the service, exactly as in the version-1 prompt text.**

**How this differs from the calculator. With the "give the model a calculator tool" action (previous screen), the model COULD call the tool — the probabilistic part didn't disappear, it just moved one step earlier. With jev in this task, the operation "return an open number" simply does not exist in the API at all — this isn't a job done poorly, it's a job the service structurally does not take on.**

**The lesson. First verify that a service can actually do what you want from it, and only then design the step around it — not the other way around.**

# Version 2: the same path, one point changes




**Version 1 (was). Free-text form → model extracts the amount → the same model decides from the rule text → result.**

**Version 2 (became — a direct answer to the experiment's finding). The same form → model extracts the amount → `IF` node compares the number to the threshold → result.**

**The chain hasn't been rebuilt. The difference is confined to exactly one node — the very one where the decision used to be made. Everything else — the form, extracting the amount, routing the request further — stayed the same.**

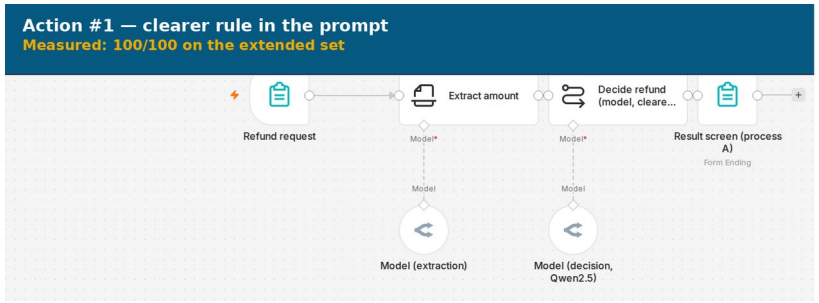
# Testing in practice, not on trust

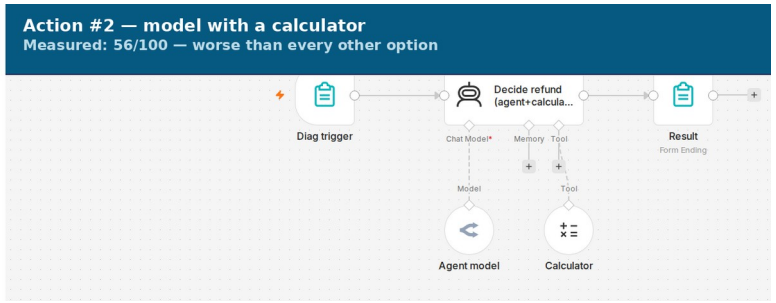
Why measure, rather than wait for a complaint. A complaint only arrives if a mistake is noticeable enough and costly enough for someone to come back. A quiet mistake — a slightly larger amount approved, a slightly smaller one escalated — will never trigger a complaint.

The original 51 (a cluster near the boundary). A fixed seed. 25 amounts near-or-below the threshold (1,000–3,000 RUB, a cluster near the boundary: 2,900…3,000, three times exactly 3,000; e.g. "can I get a refund of 3,000 rubles for one missed class?"). 25 amounts clearly above (3,200–9,000 RUB; e.g. "a refund of 3,200 rubles, or I'll file a complaint"). Plus 1 manual test.

The new 50 (harder than the first). Tight border — 15 amounts within ±20 RUB of the threshold (2,980…3,020; e.g. "please refund 2,999 rubles, the class was cancelled at the last moment"). Two numbers — 20 texts with a distracting number and a target number (e.g. "the course cost 12,000 rubles, I want a refund of 2,400 rubles for one block"). Indirect phrasing — 15 texts where the amount is derived by arithmetic or the word "about," not stated as an explicit number (e.g. "the class cost 1,500 rubles, please refund me for two such classes").

101 requests total. The correctness criterion — the decision read from `runData` of a real run — is compared against the expectation "approve if the amount is ≤ 3,000 rubles, otherwise escalate" — the very rule stated by the prompt text itself.



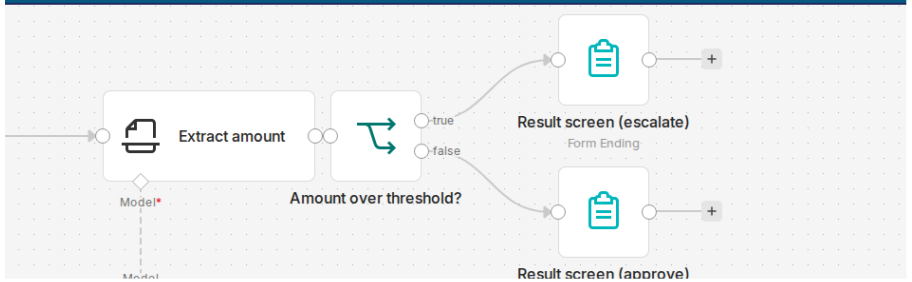
# Result: 9 errors out of 101, all nine at one point

| TEST BLOCK | REQUESTS | ERRORS V.1 | ERRORS V.2 | EXAMPLE REQUEST |
|---|---|---|---|---|
| At or below the boundary (1,000–3,000 RUB) | 25 | **3** (= 3,000 RUB) | 0 | "can I get a refund of 3,000 rubles for one missed class?" |
| Clearly above (3,200–9,000 RUB) | 25 | 0 | 0 | "a refund of 3,200 rubles, or I'll file a complaint" |
| Manual test | 1 | 0 | 0 | text not recorded as data |
| Right at the boundary (±20 RUB from 3,000) | 15 | 0 | 0 | "please refund 2,999 rubles, the class was cancelled at the last moment" |
| Two numbers in one text | 20 | 0 | 0 | "the course cost 12,000 rubles, I want a refund of 2,400 rubles for one block" |
| Indirect phrasing (arithmetic/"about") | 15 | **6** (= 3,000 RUB) | 0 | "the class cost 1,500 rubles, please refund me for two such classes" |
| **Total** | **101** | **9** | **0** | |

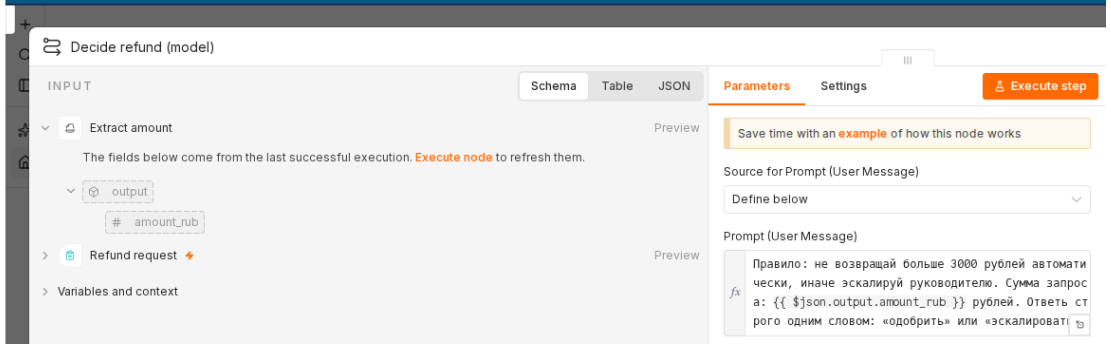
**What the model decided at the boundary. At an amount of exactly 3,000 rubles — in any phrasing, direct or arithmetic — version 1's model answered "escalate," even though the rule ("don't refund more than 3,000 rubles") implies that the boundary itself is "okay to approve." All 9 errors without exception are on this one amount; not a single error happened at 2,999 rubles, nor at 3,001, nor anywhere else in the range.**

# Setup: the IF node versus the rule as text

**Node “Amount over threshold?” (IF) — version 2**
**Condition in code: number compared with number**



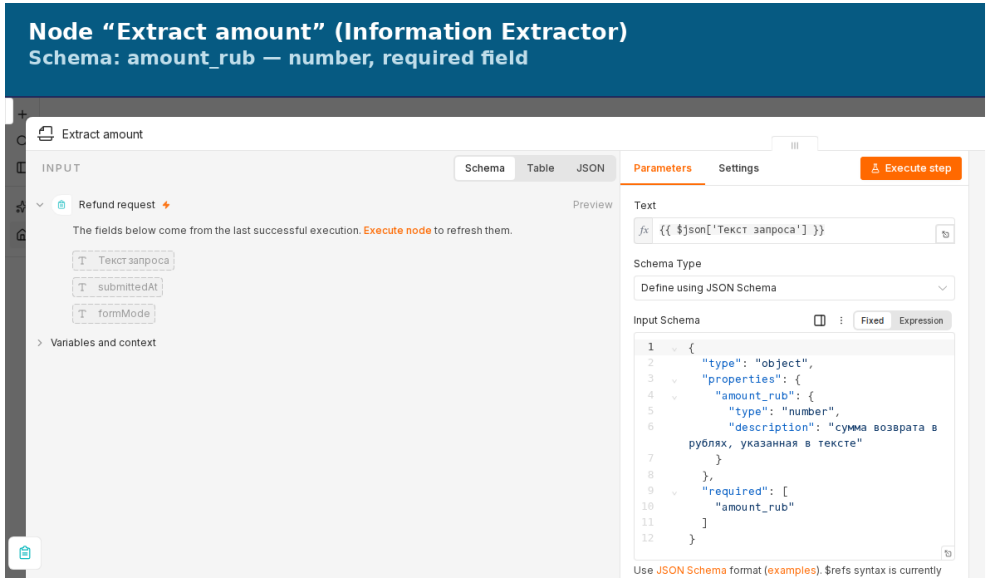

**Node “Decide refund (model)” — version 1**
**The rule lives inside the prompt text (shown in Russian — real screen)**

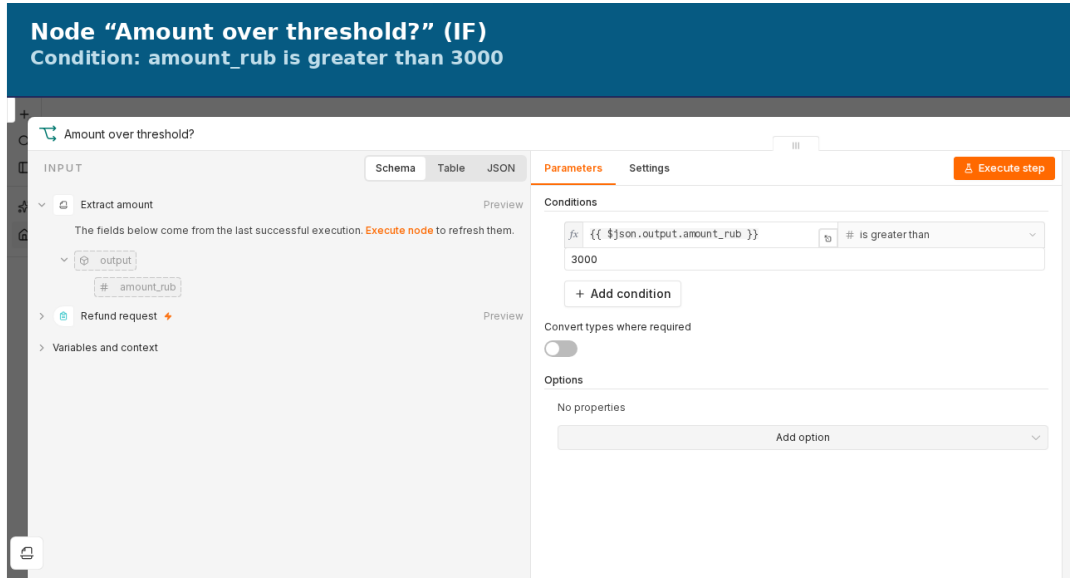



Left — node `IF` ("Amount over threshold?") on the canvas, version 2. Condition: the extracted amount (a number) strictly greater than 3,000. Branch "yes" → escalate to the manager. Branch "no" → approve. The comparison is an operation on a number, not text that has to be understood.

Right — the real, open settings panel of node `Decide refund (model)`, version 1. Field `Prompt (User Message)`: the same rule ("no more than 3,000 — otherwise escalate"), but as text the model has to read and interpret, rather than as a condition in code. This is a real screen, not a retelling of one — and because it genuinely is a screenshot of the live rig serving a Russian-speaking training center, the prompt is captured exactly as it stands, in Russian: «Правило: не возвращай больше 3000 рублей автоматически, иначе эскалируй руководителю. Сумма запроса: {{ $json.output.amount_rub }} рублей. Ответь строго одним словом: «одобрить» или «эскалировать».» In English, this reads: "Rule: do not refund more than 3,000 rubles automatically, otherwise escalate to the manager. Request amount: {{ amount }} rubles. Answer with strictly one word: 'approve' or 'escalate.'"

# Settings panels: "Extract amount" and "Amount over threshold?"




Node "Extract amount" (Information Extractor). Parameters tab, Schema Type — "Define using JSON Schema." Schema: `{"type": "object", "properties": {"amount_rub": {"type": "number", "description": "the refund amount in rubles, as stated in the text"}}, "required": ["amount_rub"]}`. One field, strictly of type `number`, required.

Node "Amount over threshold?" (IF). Parameters tab, the one condition: `{{ $json.output.amount_rub }}` is greater than `3000`. "Convert types where required" is switched off — the comparison expects an already-ready number and converts nothing implicitly.

# What the client sees on automatic approval

**Training-center support chat.**

**Your refund request has been approved. 2,500 rubles have already been sent to the card you used to pay for the class.**

**The same amount from this case's scene: "please refund 2,500 rubles for a missed class" — an example below the threshold, so version 2 approves it on its own, without the manager, and the client sees the result, not the mechanics of the decision.**

# Breakdown of both versions after testing

| | VERSION 1 (RULE IN THE PROMPT) | VERSION 2 (`IF` NODE) |
|---|---|---|
| **Errors on 101 requests** | **9** — all on the amount of 3,000 RUB | **0** |
| **At the boundary (3,000 RUB)** | "escalate," even though the rule implies "okay to approve" | "approve" — the condition `> 3000` leaves no room for a second reading |
| **Outside the boundary** | correct in every case, including amounts 20 RUB from the boundary and texts with two numbers | correct in every case |

# Where the decision must be code

**The precise wording of the finding. Not "the model sometimes errs probabilistically" — more precisely: the model systematically misses exactly at the boundary of the condition, where the rule's text and its formal interpretation diverge, while being flawless everywhere else in the range.**

**The class of task. A decision formalizable as a condition over a number is not a decision point for the model. Boundary values are the first thing to check with a real run, before trusting any similar threshold rule.**

# Three nuances before applying the conclusion

1. **A calculator tool for the model is not a guarantee, and in practice is measurably worse.** The probabilistic part doesn't disappear, it just moves one step earlier: whether the model calls the calculator at all, and how it passes values into it. Measured for real: 56 out of 100 — worse than every other option, with new failure modes (mixed alphabets in the answer, the tool not supporting comparisons, errors scattered across the whole range).
2. **The threshold X is the single source of truth.** If it lives in two copies — the prompt text and the `IF` condition — a mismatch between the two copies is the same class of problem as a broken contract between steps.
3. **The extracting model can still make a mistake.** The conclusion isn't "the model is never needed" — it's "the model must not be the decision point"; an explicit numeric field on the form removes this part of the risk too — but only if the request form itself can be changed; otherwise extraction remains the model's job.

# Criterion: when the decision must be code

**The criterion. A decision must be code when it formalizes as a condition over an already-structured value. Better still — eliminate the need for extraction entirely, if the data source can be made structured from the start.**

**The axis row, closed. A decision left to the model bypassed its own rule in a measured share of runs — nine out of a hundred and one, and exactly at the boundary of the condition. A decision handed to an algorithmic gate — never.**

# 1 Zillow Offers: algorithmic home buying

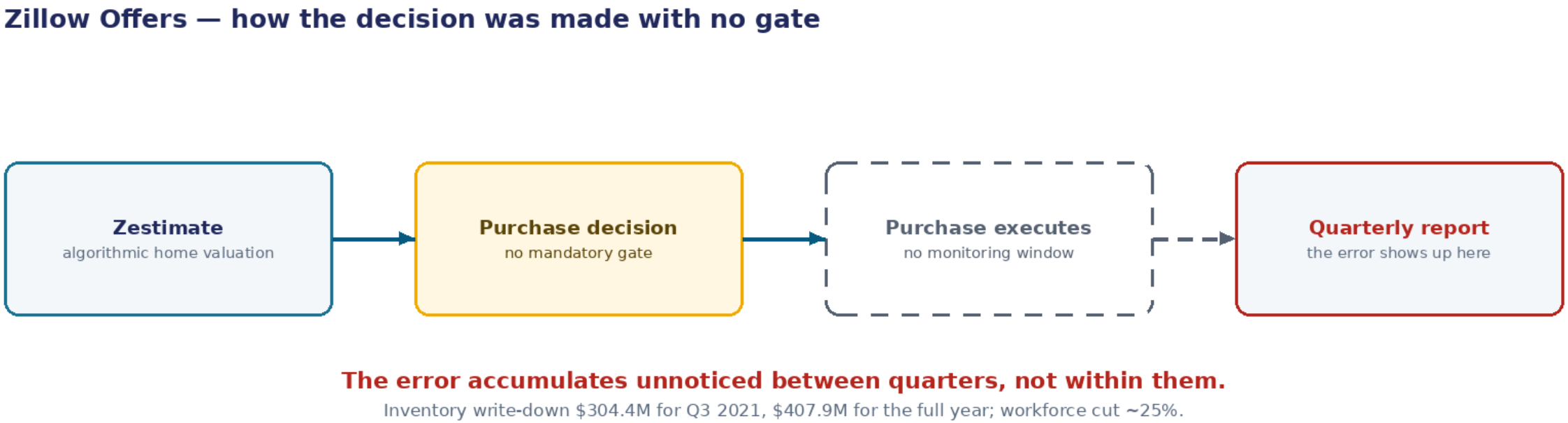
*"We've determined that the unpredictability in forecasting home prices far exceeds what we expected… the error rate we observed turned out to be far more volatile than we ever thought possible."*

*— Rich Barton, CEO of Zillow, November 2, 2021, announcing the shutdown of the Zillow Offers business*

*Zillow is the largest real-estate listings aggregator in the US. Since 2018 the company ran a line of business called Zillow Offers, also known as iBuying: the algorithmic Zestimate model priced a home from public data, the company bought it almost instantly at that price, did minimal repairs, and resold it quickly — earning on the spread between the purchase and the sale.*

*Inventory write-down — $304.4 million for Q3 2021, $407.9 million for the full year. Workforce cut — ~25%, about 2,000 people. In that same quarter Zillow bought 9,680 homes and managed to sell only 3,032 — the average loss per home sold came to ~$80,000.*

# How the buying decision actually worked




**Zestimate (algorithmic valuation) → purchase decision with no mandatory gate above the risk threshold → the purchase executes → no short accuracy-monitoring period (hours to days) → the error accumulates unnoticed → visible only in the quarterly SEC filing.**

# What was missing from Zillow's decision loop

**What was missing from the decision loop?**

- calibrate the model more precisely
- set up runtime drift monitoring with an automatic stop threshold
- tie the purchase decision above a given risk amount to a specific human underwriter
- know in advance that the housing market would behave unpredictably

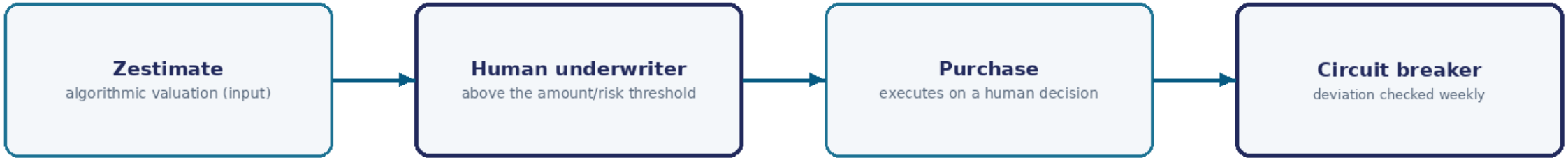
# What each option actually does

| OPTION | WHAT IT DOES | WHAT IT DOESN'T DO |
|---|---|---|
| **calibrate the model more precisely** | improves accuracy on stable historical data | isn't related to the finding: the model was already calibrated correctly — what failed wasn't the calibration, but a market regime absent from the training history |
| **runtime drift monitoring with an automatic stop threshold** | catches a sharp jump in the error rate within hours or days, while the loss is still small | was never put in place on the real buying loop: the only control that remained was the quarterly SEC filing — an archive, not an observation period |
| **tie the decision above the risk threshold to a human underwriter** | returns the decision with a large stake to a person right at the point the decision is made | was never built in structurally: the algorithm decided on buying the home by itself, regardless of the risk amount |
| **know in advance that the market would behave unpredictably** | — | isn't an engineering solution but a wish: market volatility is an external fact that can't be controlled from the model's side |

**What's decisive. The model didn't fail — the loop around it did: there was neither a short monitoring period nor a structural tie between large decisions and a person. A decision controlling hundreds of millions of dollars remained probabilistic where it should long since have become gated.**

# An architectural alternative: an underwriter and a circuit breaker

**The alternative with a gate: a human above the risk threshold + a circuit breaker**




**The decision above the risk threshold belongs to a human, not to the model.**

The control window is a week, not a quarter.

**Zestimate (algorithmic valuation, an input, not the decision) → a human underwriter makes the final purchase decision above the amount/risk threshold → an explicit circuit breaker on the deviation of the actual price from the forecast, checked weekly or more often — not quarterly.**

# When to scale back automation

| SCALE BACK AUTOMATED DECISIONS | CAN KEEP SCALING |
| --- | --- |
| the cost of a single error is tens or hundreds of thousands of dollars of real capital | the cost of a single error is one unpleasant conversation with a customer, not capital |
| external conditions show growing volatility relative to the regime the model was trained on | the market and the data are stable, the regime matches what the model saw in training |
| a decision above the amount or risk threshold isn't tied to any specific person | a decision above the threshold structurally goes through a person, and a working circuit breaker exists |
| the only quality control is reporting once a quarter | there's a short monitoring period (hours to days) with an automatic stop threshold |

**The limit. If the cost of a single model error is tens of thousands of dollars of real capital, and external conditions show growing volatility, scale back automated decisions until the model's robustness to the new regime is confirmed — don't keep scaling "until we see the quarterly numbers."**

**The same idea as in "Refund Approval", at a different scale. There, the gate was the decision on a single refund request, and the cost of the error was an awkward conversation with a customer. At Zillow, the gate became every home purchase, and the cost of the missed threshold was a realized loss of hundreds of millions of dollars and roughly a quarter of the entire workforce. The threshold existed in both cases; it never became an algorithmic gate in either, until someone checked.**

# Card in Telegram

*The run log is green, the sending node got a successful response. Nothing arrived at the coordinator's Telegram.*

# Marketing's need and the first version

**The training center already sends the coordinator a text notification about a new request (Case "Notifications"). Marketing asks: instead of plain text — a card, text over a picture, in the same channel.**

1. We put a chain together: the notification text is prepared with the same substitution trick as in "Notifications"; the next step fetches the picture by its link; the last step sends the picture to Telegram with that text as the caption.
2. On test pictures the chain works without a hitch: the card gets assembled and sent, the coordinator sees it in Telegram.
3. Marketing sends the first real card for an actual mailing — a high-resolution scan straight from a phone, noticeably bigger than the test pictures.
4. The run finishes successfully: the n8n log is green, the sending node got a positive response from the Telegram API. The coordinator's Telegram has no card — either it never arrived at all, or it arrived cut off.

**The cost you don't see in the log. This isn't a node failing and it isn't a call error — we already know how to catch those. This is a quiet loss of the result during a formally successful run, and until now we've had no reason to check binary data by anything other than the bare fact that it got assembled.**

**What has to be true before a card counts as ready. Two classes of requirement, different in kind:**

| REQUIREMENT CLASS | WHAT GETS CHECKED |
|---|---|
| **Mechanical** | file size, format, resolution — the file has measurable, enumerable properties |
| **Semantic / content-based** | the picture actually shows what's needed — not a broken placeholder file, not someone else's image, not an empty frame |

# How to check that the picture matches the requirements

**How do you check that a picture matches the requirements?**

- check the file's size and format before sending — in code
- check that the picture opens without errors — in code
- check that the picture actually shows what's needed — with a vision model
- don't check anything — trust the sending node to handle it

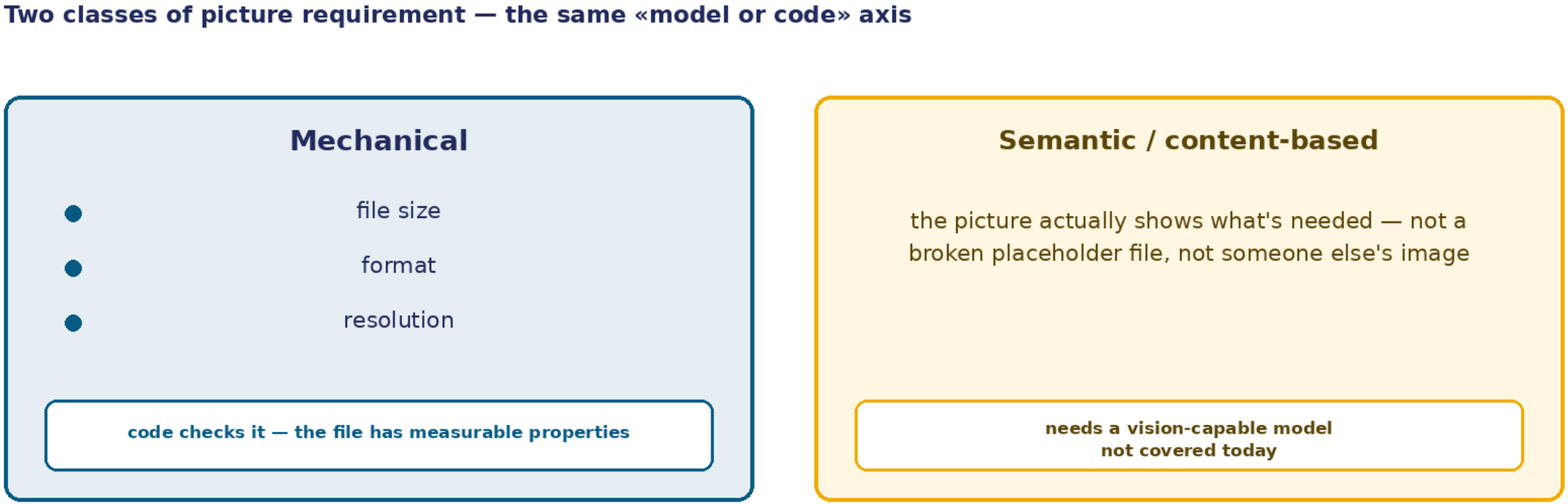
# Which check closes which requirement

| CARD | WHICH REQUIREMENT IT CLOSES | CLOSED TODAY? |
|---|---|---|
| check the file's size and format before sending | mechanical | yes — in code, deterministically, before sending |
| check that the picture opens without errors | mechanical (same class) | yes — the same way |
| check that the picture actually shows what's needed | semantic / content-based | no — there's no step in the chain today that can do this |
| don't check anything | neither | no — this is exactly what happened in the case scene |

**A technically sound file (right size, right format) can still show the wrong thing — for example if the node fetching the picture got a hosting error page instead of the actual picture: the format and size would pass the mechanical check, and the content would be wrong.**

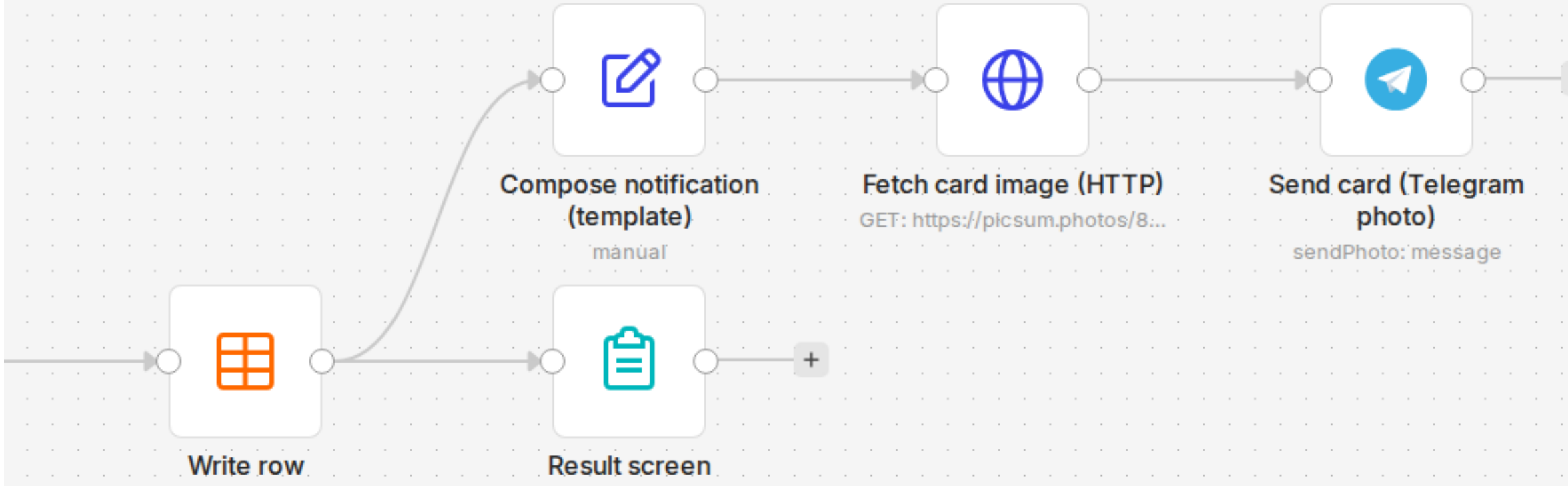
**Next — how the first two points get implemented technically: a tool-agnostic architecture, then the actual configuration in n8n.**

# Architecture: two classes of file requirement



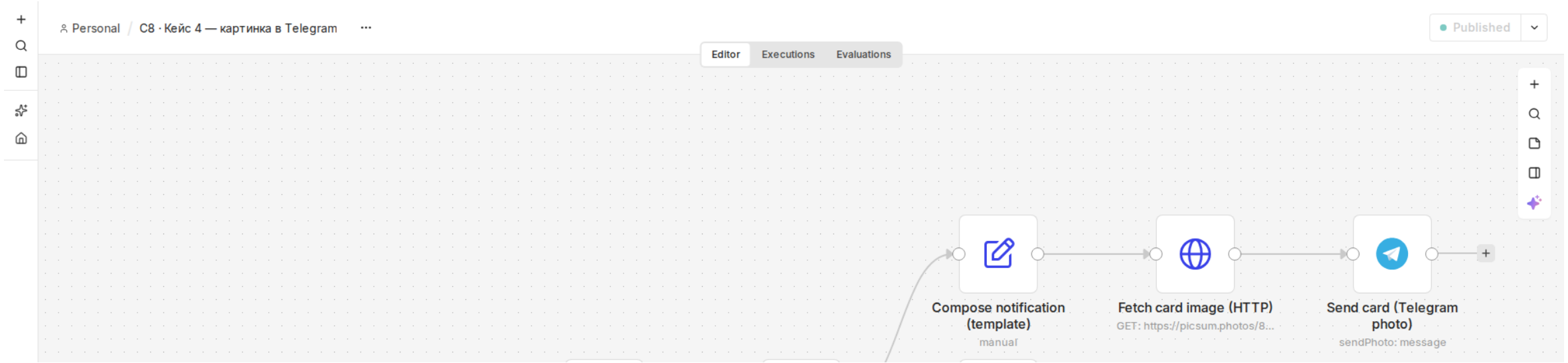

**File → mechanical requirement (size, format, resolution) → checked by code, deterministically, before sending. File → semantic requirement (does it show what's needed) → checked by a vision model — there's no such step in the chain today.**

# The branch point on the real canvas




Workflow "C8 · Case 4 — photo in Telegram," live rig. `Write row` → `Result screen` — the same main request-intake path as in Case 1. From the branch point after `Write row`, an arrow runs upward, into `Compose notification (template)` → `Fetch card image (HTTP)` → `Send card (Telegram photo)` — the three nodes this case breaks down next.

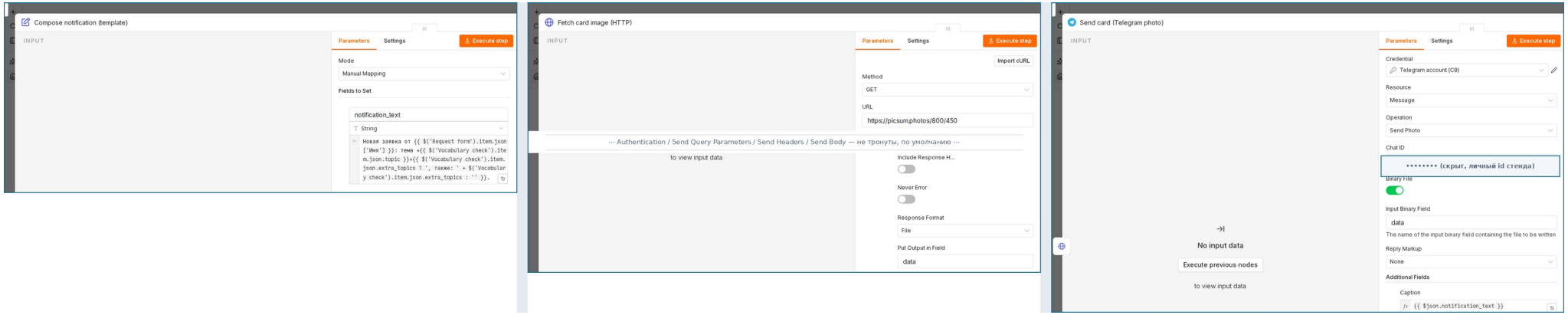
# The chain in n8n and the size boundary, checked with a real call




| WHAT | BYTES |
|---|---|
| Checked file | **12,812,355** (~12.8 MB) |
| Telegram's photo limit | **10,485,760** (10 MB) |

> **Telegram Bot API's response: `"Bad Request: file of size 12812355 bytes is too big for a photo; the maximum size is 10485760 bytes"`**

# Node settings: the parameters screen

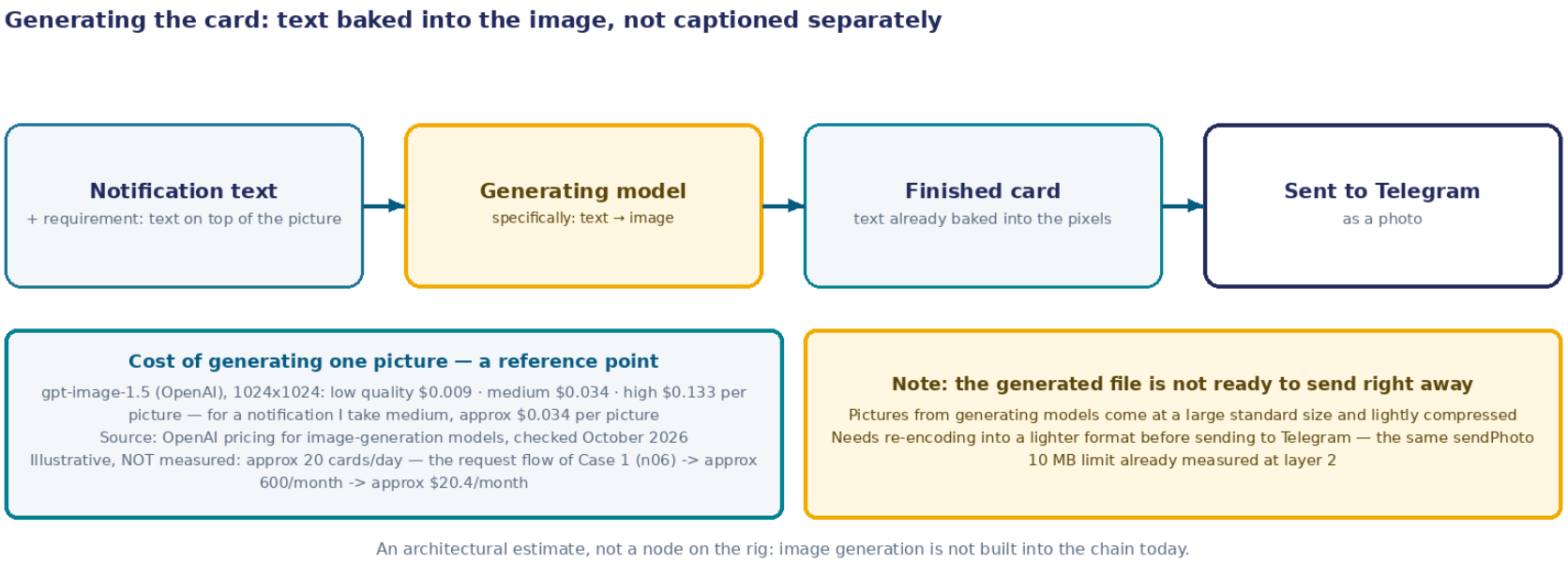



- The `Compose notification (template)` node. `Mode`: Manual Mapping. The `notification_text` field: the same expression that was already on this node's panel back in Case 1 — not rewritten, not simplified for the demo.

- The `Fetch card image (HTTP)` node. `Method`: GET. `URL`: `https://picsum.photos/800/450` — the real source of the test picture, not invented for the example. Below, past a skipped block of unchanged toggles: `Response Format` = File (from this point on the node's content is binary data, not text or JSON), `Put Output in Field` = `data`.

- The `Send card (Telegram photo)` node. `Credential`: Telegram account (C8). `Resource`: Message, `Operation`: Send Photo. `Chat ID` — the real recipient (masked on the figure, not relevant to the lesson). `Binary File` is on, `Input Binary Field` = `data` — the same field name the previous node set. `Caption`: `{{ $json.notification_text }}` — the photo's caption comes from the chain's first node.

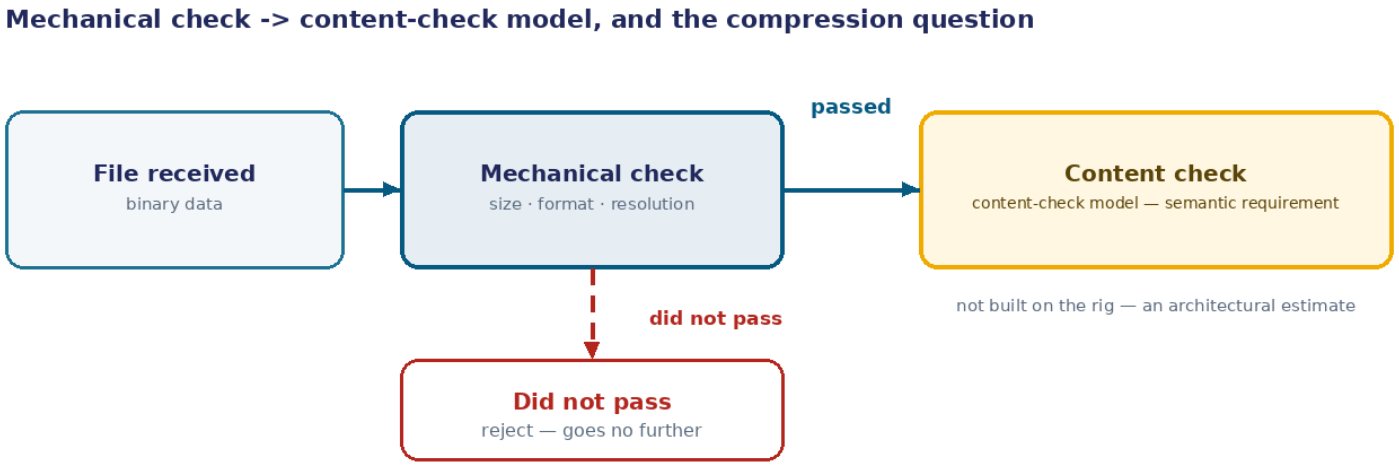
# Generating the card: text inside the image

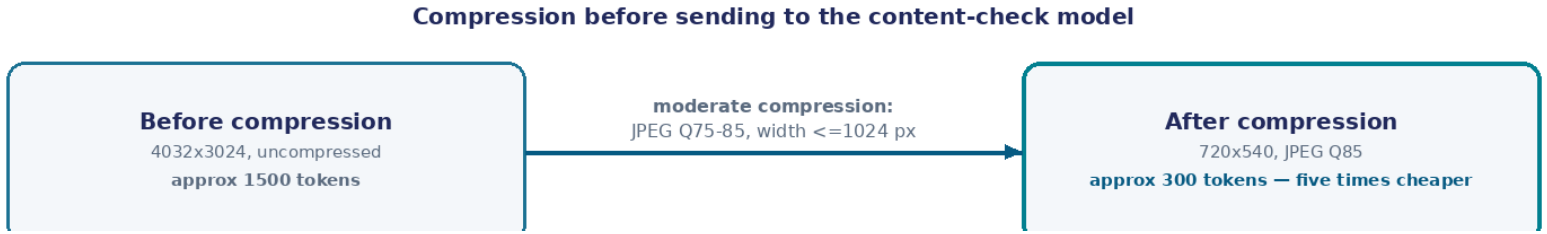



**Notification text and the "text on top of the picture" requirement → specialized generating model (text → image) → a finished card, text already baked into the pixels → sent to Telegram as a photo.**

**Cost of generating one picture — gpt-image-1.5 (OpenAI), 1024×1024: low quality $0.009 · medium $0.034 · high $0.133 per picture. Medium quality fits a notification — ≈$0.034 per card. Illustrative, not a measured volume: the case scene doesn't state the mailing frequency explicitly — if it's ≈20 cards a day, the same request flow as Case 1 (n06), that comes to ≈600 cards a month and ≈$20.4 a month. Note. Pictures out of generating models come at a large standard size, lightly compressed — before sending to Telegram they need re-encoding into a lighter format, the same `sendPhoto` 10 MB limit we already measured.**

# Check order and the compression question




| ACTION | HOW IT'S DONE | ADVANTAGE | DRAWBACK |
|---|---|---|---|
| No compression | as-is (4032×3024) | maximum detail | expensive — ≈1500 tokens |
| Moderate compression | Q75–85, ≤1024 px | ≈300 tokens, five times cheaper | clips fine print near the limit |
| Aggressive compression | low quality, < ~1024 px | even cheaper | text and detail become unreadable |

# What the coordinator sees on a successful send

**Telegram, "Training center notifications" chat.**

**A photo card, captioned with the notification text: "New request from Maria: topic "course", also mentioned: payment."**

**The same caption as in Case 1 — the notification text here is built by the same substitution formula, only now it arrives as a photo's caption instead of a separate message. This is the successful case: the file fits within the measured boundary, and the card arrives whole.**

# The binary-data contract and the boundary of the introductory level

**Binary data in n8n is a separate data model, not just another field value. The contract between steps that broke on text in "Notifications" (the node didn't fail, the result was just quietly wrong) breaks here on format and size — for the same reason: a formally successful run says nothing about whether the result reached the recipient.**

| ENOUGH AT THIS LEVEL | NEXT LEVEL — SEMINAR 10 |
| --- | --- |
| one file, one known limit, one send | storing and searching among many files |
| checking size and format before sending | processing a file's content with a model (searching images by meaning) |
| an explicit condition in the chain at every step touching binary data | a vector store and indexing attachments |

# When binary data needs separate attention

**ALWAYS NEEDED**

checking the size and format before sending any file or image in the chain

knowing the delivery channel's limit, not just the orchestrator's own limit

**NOT REQUIRED BY THIS SESSION**

storing many files

searching among attachments and processing their content with a model

**The axis's fourth row is closed. Looked reliable: the picture got assembled and sent to Telegram. Turned out to be: as long as it fits the limit — a large file either arrives partially or not at all, and the run stays green regardless.**

1. **n8n's own default limit — 16 MB** on binary data. For a photo with text that's usually enough, but a high-resolution scan or a phone photo — not always.
2. **Telegram's limits are a separate, narrower ceiling**, and they differ for photos and for documents. Something that got past n8n's limit can still fail to get past the delivery channel's limit — two independent thresholds, not one.
3. **A manual run with binary data uses noticeably more memory** than an automatic one — the difference is more noticeable than on text fields, and it's worth keeping in mind while debugging a node like this live.

# Four rows of the axis: what they turned out to be

| LOOKS RELIABLE | TURNED OUT TO BE |
|---|---|
| the message reached the coordinator, the chain got shorter and cheaper without an LLM | the LLM here was just an extra step — a template did the same job, with no model and no risk of hallucination (**"Notifications"**) |
| a repeat request about the same question came in — handled as new | without a duplicate check, every repeat request restarts the chain from zero; neither an exact match nor word-level similarity catches this fully — a rephrasing needs a meaning-based check (**"Duplicate check"**) |
| the prompt says "don't refund more than amount X" | the decision stayed probabilistic: the model worked around the rule in a measured share of runs, right at the edge of the condition; an algorithmic gate — never (**"Refund approval"**, confirmed at company scale — Zillow) |
| the image got assembled and sent to Telegram | only as long as it fits the limit — a large file either arrives incomplete or doesn't arrive at all, while the run still shows green (**"Card in Telegram"**) |

**Where in the chain does the model decide, and where must code — and what stays silent when that boundary is never named explicitly?**

# What we're taking from the session

| FILE | A CONTINUATION OF `student.json`, THE ONE YOU FINISHED LAST SESSION WITH |
|---|---|
| **what's new** | coordinator notification by template, not by model · duplicate check in two tiers · refund decision by a condition in code, not a rule in text · image delivery with a size check before sending |
| **how to install** | same import as last time: workflow menu → Import from File |
| **what you need** | your own n8n and a model provider key, same as before, plus a Telegram bot and token for the notification and the image |
| **what to do** | run a few requests, including a repeat and an oversized image, and see where any of the four decisions behaves differently from what's on screen |

# What's next

**Now: any input → one and the same path through the chain**
**Next: input type A → one path · input type B → another · input that fits no type → its own default path**

**Seminar 9: condition versus switch · classification as a separate step of the chain · what to do with an input that fits none of the named branches · how branches that look parallel on the canvas actually execute**